Understanding How Learning Rate and Number of Trees Affect Performance in XGBoost

Name : Vineeth Doddaballapura Raju

Student No : 24089093

1. Introduction

Gradient Boosting is a highly effective machine learning technique with vast practical implications. Its application is diverse and ranges across finance, healthcare, and recommender systems. This is because of its robust predictive accuracy. Techniques such as XGBoost, LightGBM, and CatBoost are built on top of gradient boosting.

The objective of this tutorial is to help readers understand the operational mechanics of Gradient Boosting, its strengths, and how it can be used in individual project. This tutorial focuses specifically on the relationship between learning rate and number of estimators in XGBoost. Rather than only describing the theory, we demonstrate this trade-off experimentally using real-world data.

2. Key Idea Behind Gradient Boosting

Gradient Boosting is an algorithm for building prediction models through an ensemble of multiple models. The models used in this algorithm are decision trees. The algorithm combines multiple decision trees in an iterative fashion to create an effective prediction model. The algorithm is not based on building one effective model. Instead, it starts with an effective model and makes it better. In each step it calculates the predictions made by the model, finds the errors in predictions, and trains another model to predict those errors. Then, it combines this new model with the previous model to improve predictions. In this way, it makes incremental improvements in predictions in each step.

3. Mathematical Intuition

Gradient Boosting can be viewed as an optimization algorithm that seeks to optimize a loss function through iterative improvements to the predictions. While gradient descent optimizes the parameters directly, Gradient Boosting optimizes in the space of functions by adding models that are an estimate of the negative gradient of the loss.

In each iteration, the algorithm estimates the error between the predictions and the actual values based on a chosen loss function, typically Mean Squared Error for regression problems. Then, the gradient of the loss with respect to the predictions is computed. This gradient represents the errors made by the dominating model in the ensemble.

This view explains why Gradient Boosting can solve problems with high accuracy. The successive model does not have to learn the problem again; instead, they learn to reduce the error made by the previous model. Through repeated iterations, the accuracy of the model is very high.

4. Bias–Variance Trade-off

The bias-variance tradeoff describes the balance between underfitting and overfitting. Bias occurs when a model is not sufficiently simple and is unable to recognize patterns in the data, while variance occurs when a model is overcomplex and inadvertently picks up noise in the data.

Gradient Boosting helps in reducing bias by adding models that correct previous errors. This helps in learning complex relationships. However, adding an extra number of trees or making them deep can cause overfitting and hence increase variance. The learning rate and depth need to be controlled.

- **High bias (underfitting):** Model is too simple → poor learning
- **High variance (overfitting)**: Model is too complex → learns noise

5. Key Hyperparameters

The quantity of estimators represents the quantity of trees used by the model. Increasing this quantity can improve performance but also increase overfitting. The learning rate controls how much each tree contributes to the model's performance. Lower values improve generalization performance. The maximum depth controls how complex each tree is. More depth can improve detail, but increase overfitting.

6. Focus: Learning Rate vs Number of Trees

The key concept discussed in this tutorial is the interaction between the learning rate and the number of trees. These are the key parameters in determining how the model learns from the data.

If the learning rate is low, the contribution of each tree to the model is reduced. This implies that more tree are required to ensure good performance. This results in a slower learning process with better generalization.

On the other hand, if the learning rate is high, each tree contribute more to the model. This implies that the model can perform better with fewer tree, but the risk of overfitting also increases.

Low learning rate → small updates per tree → needs more trees → stable learning → better generalisation
High learning rate → large updates per tree → fewer trees needed → faster learning → risk of overfitting

7. Practical Example: Predicting Apple Stock Movement

To demonstrate this concept, Apple stock price data from 2020 to 2024 was used. Several features were engineered, including daily returns, moving averages, and volatility, which capture different aspects of market behaviour.

The problem was treated as a binary classification problem, where the task is to predict whether the stock price will rise or fall on the next day. Time series splitting was used to maintain the sequence in the data.This is to ensure that the evaluation of the model is realistic.

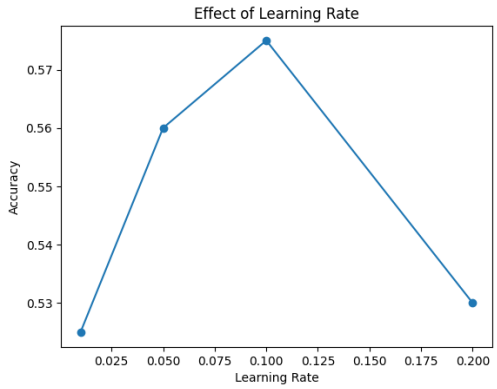
8. Effect of Learning Rate

In the learning rate experiment. The learning rate used were 0.01, 0.05, 0.1, and 0.2. All other parameters were kept constant. This experiment showed that the learning rate is too low when it is 0.01 and that the model underfit the data.

On the other hand, the best result are obtained when the learning rate is 0.05 or 0.1. This is because the model is able to learn something without overreacting to the noise. Learning rates such as 0.2 caused performance to degrade. This is because they caused overfitting.

- Small values → slow learning, better generalisation

- Large values → fast learning, higher overfitting risk



9. Effect of Number of Trees

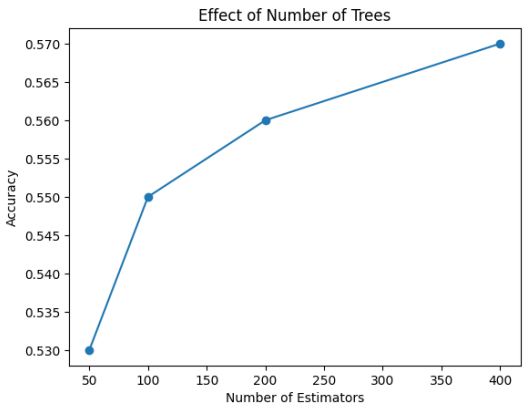
The number of trees experiment tested models that used 50, 100, 200, and 400 trees. From this, it was seen that when too few tree were used, such as in a model that used 50 trees, underfitting was a problem.

When more trees were used, such as in models that used 100 and 200 trees, a significant improvement was seen, as the model was able to fine-tune its predictions.

When more trees than that, or 400, were used, it was seen that there was a point of diminishing returns, and overfitting was a problem.

Observations:

- **50 trees → Underfitting**
- **100–200 trees → Optimal**
- **400 trees → Diminishing returns / overfitting**



10. Combined Interpretation

The combined result show that a proper balance between these two factors is required for optimal performance. A low learning rate requires more trees, and a high learning rate requires fewer trees but can cause overfitting.

The best performance is obtained using a moderate learning rate and a sufficient number of trees, which is consistent with theory. This is one of the most important ideas in Gradient Boosting and is a critical concept for effective Gradient Boosting.

Low learning rate + many trees → gradual learning → strong generalisation
High learning rate + few trees → fast learning → unstable model

11. Model Evaluation

The model achieved an accuracy slightly above baseline, which is expected given the difficulty of predicting stock price movements in noisy financial markets .

The classification report provides insight into precision, recall, and F1-score, while the confusion matrix highlights where the model makes errors

Feature importance analysis shows that recent returns and moving averages have a stronger influence on predictions than volatility.

12. Practical Tips

In practice, it is recommended that the learning rate be set to a moderate values, such as 0.05, and the number of tree be set to a range of 100 to 200.

If the model underfit, the number of tree may be increased to aid in the learning process. If the model overfits, the learning rate may be reduced to stabilise the learning process.

Cross-validation may also be used to identify the optimal combination of hyperparameters more effectively.

13. Advantages

- High predictive accuracy
- Capable of modelling complex non-linear relationships
- Handles a variety of data types
- Performs well with minimal feature engineering
- Widely used and effective in real-world applications

14. Disadvantages

- Computationally expensive, especially for large datasets
- Sensitive to hyperparameter tuning
- Risk of overfitting if not properly controlled
- Slower training compared to simpler models

15. Conclusion

Gradient Boosting is an effective and versatile machine learning method with high performance through iterative error correction. The learning rate is crucial in controlling how quickly the model is able to learn, while the number of trees is essential in controlling how much the model is able to learn.

The interrelation between these two parameter is critical in attaining an optimal performance. By understanding this interrelation, it is possible to develop an accurate model.

XGBoost is an effective solution for prediction problems when it is properly tuned; however, it is always important to take into account problems such as noisy data.

16. References

Friedman, J.H., ‘Greedy Function Approximation: A Gradient Boosting Machine’, *Annals of Statistics*, 29.5 (2001), pp. 1189–1232.
Hastie, T., Tibshirani, R. and Friedman, J., *The Elements of Statistical Learning*, 2nd edn (New York: Springer, 2009).
Scikit-learn, ‘Gradient Boosting’, available at: https://scikit-learn.org/ [accessed 27 March 2026].

17. Git-hub link

https://github.com/VineethDoddaballapuraRaju24089093/ml_assinment